

Crystal detection and classification with Detectron2 and gaussian mixture models

Raphaël Kuhn rkuhn@bordeaux-inp.fr ENSEIRB-MATMECA

September 17, 2025

Contents

1	Introduction	2
2	Dataset presentation	2
3	First approaches for crystal detection	2
3.1	Crystal Detection using YOLOv3	2
3.2	Crystal Detection using only image processing methods	2
4	Crystal detection and segmentation using Detectron2	5
4.1	Detectron2 overview and rationale for use	5
4.2	Images Pre-Processing for Detectron2	6
4.3	Evaluation metrics for Detectron2	6
4.4	Training and fine-tuning of the Detectron2 model	6
5	Non-Supervised Classification of crystals in multiple classes	7
5.1	Evaluation metrics for non-supervised classification	7
5.2	Non-Supervised Classification using Gaussian Model Mixture (GMM)	7
6	Pipeline implementation	9
7	Conclusion	10

List of Figures

1	Examples of Crystal Images	3
2	Example of crystal detection results using YOLOv3	3
3	Crystal detection results after shape detection-based image preprocessing	4
4	Example of Detectron2 Application	5
5	Comparison Between Individual and Aggregate Masks	5
6	Crystal detection results after with Detectron2 model	7
7	Exemple of crystal detection using detectron2 avoiding bubbles false-positive	7
8	AI pipeline for crystal analysis	9
9	Gradio-Based Web Interface for Crystal Image Detection and Classification	9

Abstract

We fine-tuned a Detectron2 model to detect and segment multiple crystals in microscopic images. From the resulting segmentations, we estimate crystal sizes and assign them to categories using an unsupervised Gaussian Mixture Model (GMM) classifier. These components are integrated into a streamlined pipeline, which we made accessible through an interactive Gradio-based graphical interface hosted on Google Colab.

1 Introduction

The primary objective of this project is to design a complete pipeline capable of detecting and characterizing protein crystals from microscopic images in order to improve the precision and efficiency of crystal measurement on microfluidic chips.

This pipeline must accurately identify individual crystals within images, measure their properties, and classify them into relevant categories. To achieve this, we explore advanced deep learning techniques for object detection, instance segmentation, and unsupervised classification applied to microscopy data.

First, we will present the creation and organization of the datasets used to train and test our methods. Next, we will describe the initial approaches explored for crystal detection, followed by the final approach based on Detectron2. Then, we will discuss the unsupervised classification of crystals. Finally, we will show how the project can be made accessible through a web interface.

2 Dataset presentation

The dataset used in this work was created by us. It originally consisted of 1,860 microscope images. Each image contains between 0 and approximately 60 crystals of varying sizes. The images are either in a standard rectangular format or in a circular “microscope view” format as we can see on Figure 1.

A total of 347 of these images were annotated using Labelme [1] to build the detection dataset. This dataset includes the images as well as the segmentation polygons and is divided into a training set and a test set. Des datasets plus petits contenant mois d’images annotees ont aussi ete testes.

The remaining non-annotated images were grouped into two datasets for the classification task: a training set and a test set.

Dataset Name	Number of Images
train_detect	237
test_detect	145
train_classif	676
test_classif	170

Table 1: Summary of the dataset splits.

3 First approaches for crystal detection

This section presents the initial approaches we explored for crystal detection. We discuss both the use of the YOLOv3 framework for automated object detection and traditional image processing methods based on shape analysis. These preliminary methods serve as a baseline for evaluating more advanced techniques.

3.1 Crystal Detection using YOLOv3

As an initial approach, we relied on the YOLOv3 (You Only Look Once) framework [3], a method previously developed by the team. Its main strength lies in its high processing efficiency, making it well suited for rapid object detection tasks. However, the method showed limitations in reliability and the nature of its results. Indeed as we can see on figure 2, on images containing a large number of crystals (more than around 40), the model fails to detect all of them, identifying at most about 30.

3.2 Crystal Detection using only image processing methods

We also tested a method entirely based on traditional image processing, without the use of AI. The preprocessing steps included conversion to grayscale, followed by contour dilation (see Figure 3a) to enhance visibility, and then contour detection using the Canny algorithm [4]. The detected shapes were further filtered to avoid, for instance, identifying surface reliefs within the crystals. Despite this post-processing, the method frequently detected internal structures inside the crystals and struggled

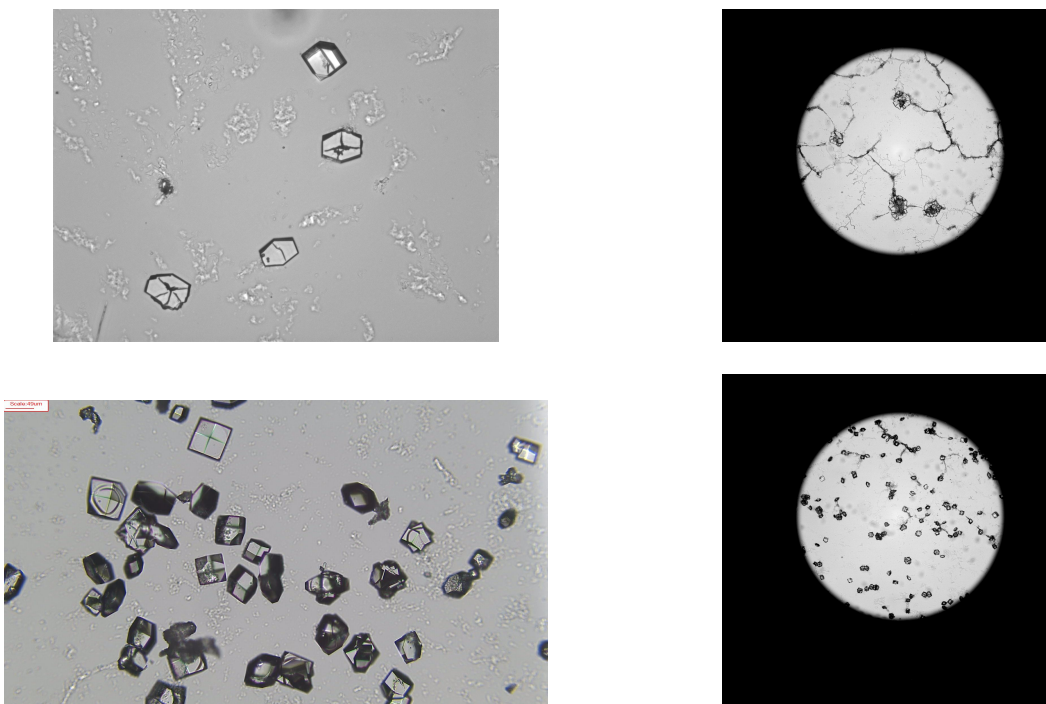


Figure 1: Examples of Crystal Images

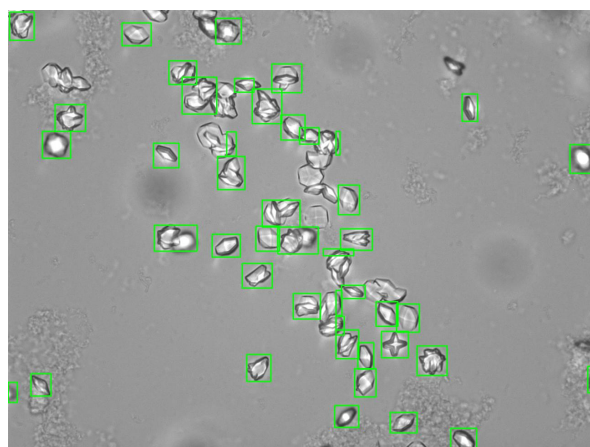


Figure 2: Example of crystal detection results using YOLOv3

when crystals were located too close to each other. As illustrated in Figure 3b, closely neighboring crystals were sometimes merged into a single detection or, conversely, split into multiple fragments.

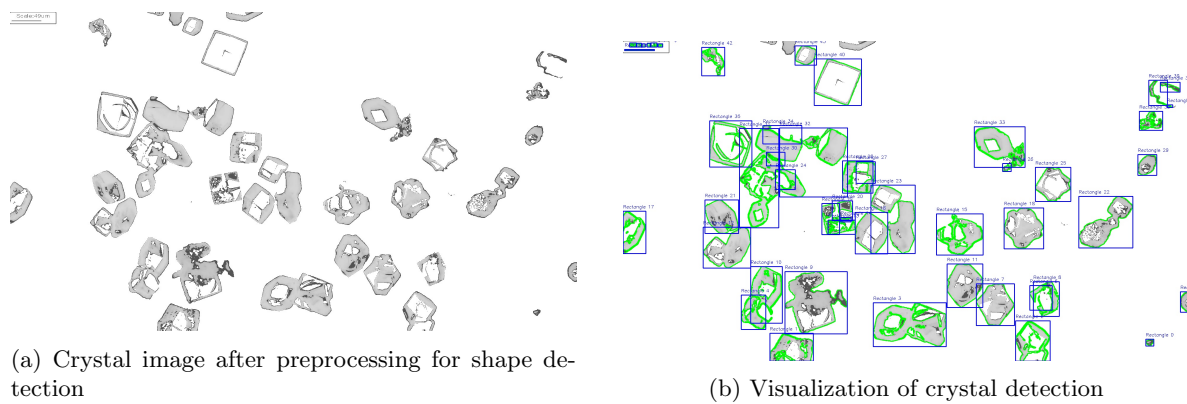


Figure 3: Crystal detection results after shape detection-based image preprocessing



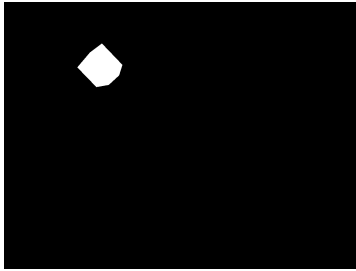
Figure 4: Example of Detectron2 Application

4 Crystal detection and segmentation using Detectron2

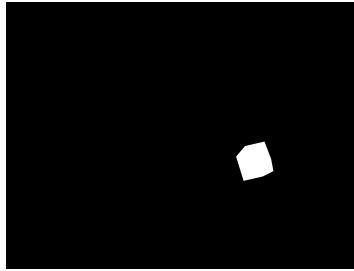
4.1 Detectron2 overview and rationale for use

We will now focus on the alternative of fine-tuning a model to detect and segment our crystals. Our criteria are that both detection and segmentation must be multi-instance, meaning that each crystal is detected individually. Indeed, we do not want a cluster of touching crystals to be considered as a single crystal. Therefore, the output of our model must be a list of segmentation masks rather than a single mask.

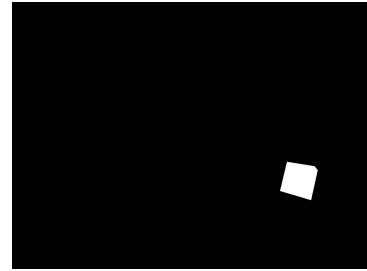
This is why we turned to Detectron2, Facebook’s solution [6], which is a library of multi-instance detection and segmentation models pre-trained on ImageNet datasets (see Figure 4). In our case, we will fine-tune a pre-trained Detectron2 model specifically on crystals. Additionally, we will use the Detectron2 Python library to handle the model throughout the fine-tuning process and during its deployment.



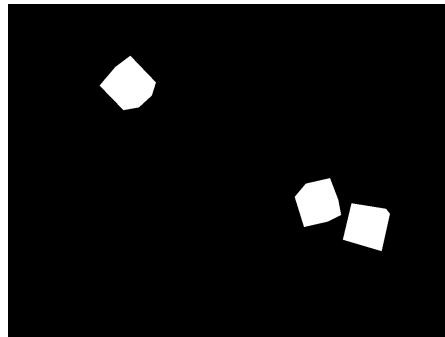
(a) Mask crystal 1



(b) Mask crystal 2



(c) Mask crystal 3



(d) Aggregate Mask of crystals 1, 2 and 3

Figure 5: Comparison Between Individual and Aggregate Masks

4.2 Images Pre-Processing for Detectron2

The pre-processing steps are directly integrated into the Detectron2 model. They consist of three stages:

- **Image Resizing:** Each input image is resized according to the default configuration of Detectron2. Specifically, the shorter side is resized to 800 pixels, while ensuring that the longer side does not exceed 1333 pixels, all while preserving the aspect ratio.
- **Image Normalization:** The pixel values are normalized using the ImageNet statistics, i.e.
`PIXEL_MEAN = [123.675, 116.28, 103.53], PIXEL_STD = [58.395, 57.12, 57.375]`
- **Tensor Conversion:** Finally, the images are converted into tensors before being passed to the Detectron2 model, which we describe in detail below.

4.3 Evaluation metrics for Detectron2

The primary goal of the model is to achieve robust crystal detection rather than highly precise segmentation. We prioritize a model that effectively identifies crystals over one that produces extremely fine boundary delineations.

To evaluate the performance of the detection and segmentation model, we use the Dice coefficient (Dice Similarity Coefficient, DSC), defined as:

$$\text{DICE} = \frac{2|X \cap Y|}{|X| + |Y|}$$

where X is the set of predicted positive pixels and Y is the set of ground truth positive pixels. Our choice of the Dice similarity coefficient over Intersection over Union (IoU) was motivated by its relative insensitivity to small boundary mismatches. As shown in [2], Dice and IoU are mathematically related, with IoU generally yielding lower values for the same overlap, thereby penalizing boundary errors more severely. Since our ground-truth segmentations are manually annotated and thus imperfect, we consider Dice a more appropriate metric for our case.

4.4 Training and fine-tuning of the Detectron2 model

For the training of our model, we adopted a configuration based on the pretrained Mask R-CNN R101-FPN architecture available in the Detectron2 model zoo. We disabled the testing dataset during training. We used 2 dataloader workers to balance speed and memory usage. Training was initialized from pretrained COCO weights to accelerate convergence. The batch size was set to 4 images per iteration (`IMS_PER_BATCH = 4`), while the learning rate was fixed at 0.00025, a standard value for fine-tuning on smaller datasets. The maximum number of iterations was limited to 1000, as preliminary experiments showed that a few hundred iterations were already sufficient to achieve convergence, although larger datasets would require longer training. Learning rate decay was disabled (`STEPS = []`) to maintain stability. For the RoI head, we used a batch size of 512 proposals per image (`BATCH_SIZE_PER_IMAGE = 512`) to improve detection quality, and finally, the number of classes was set to 1, corresponding to the single target class of crystals.

The target performance was set at 90% according to the evaluation metrics defined earlier. Since training time was not a limiting factor and the baseline Detectron2 model was already highly effective, the key parameter to reach this threshold was the number of annotated images in the training dataset. Because the annotations had to be performed manually, the process was time-consuming and laborious, and the model was retrained regularly as new images became available. It was only once the dataset reached the size presented in the final version (see Table 1) that the required average score was successfully achieved.

As shown in Figure 6, the model accurately detects the boundaries of the crystals, even when they are numerous or small in size. The detection is truly multi-instance, with each crystal identified individually, which is essential for the subsequent classification. Furthermore, the model avoids false positives, such as air bubbles (see Figure 7).

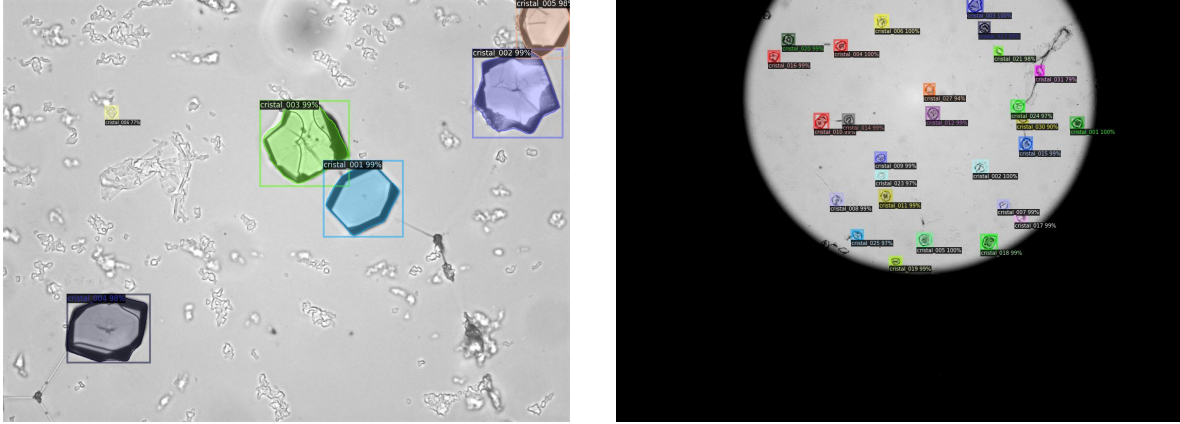


Figure 6: Crystal detection results after with Detectron2 model

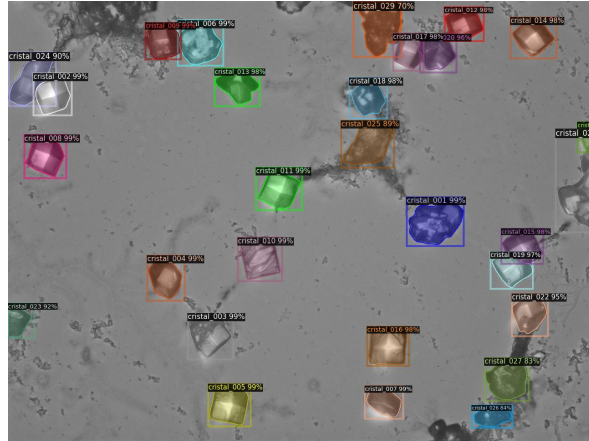


Figure 7: Example of crystal detection using detectron2 avoiding bubbles false-positive

5 Non-Supervised Classification of crystals in multiple classes

5.1 Evaluation metrics for non-supervised classification

For the classification task, we focused ourselves on non-supervised classification. Indeed our goal here was more to get an idea of how we could classify the crystals more than get a model that follows guidelines. Therefore, since it is performed in an unsupervised manner, we cannot directly compute supervised metrics such as accuracy or F1-score. Instead, we rely on internal clustering metrics and qualitative analysis of the results.

5.2 Non-Supervised Classification using Gaussian Model Mixture (GMM)

The GMM model was chosen because it is an unsupervised classification method that can be trained beforehand [7]. This ensures consistency in the definition of the classes across all images. For example, if 100 crystals are classified in a first batch and then 10 crystals in a second batch into three classes, the assignments will remain coherent since they are based on the same trained model. This is why we built a dedicated dataset for classification.

To train and evaluate our model, we extract the features produced by Detectron2 before the inter-

pretation stage. More precisely, we extract the `p3` features. We apply the same `confidence_threshold` used for detection, which ensures consistency in the selected objects.

As a result, we obtain one feature vector per crystal, which serves as the input to the GMM model. A PCA is then applied to reduce the dimensionality of the feature vectors to 64, but no further preprocessing is performed, as additional transformations could disrupt the balance learned by the Detectron2 backbone. After training, the cluster weights obtained are as follows:

Cluster weights: [0.5926, 0.1650, 0.2252, 0.0172].

We observe the presence of a dominant class corresponding to small crystals. This overrepresentation can be explained by the fact that images containing small crystals often include several dozens of them, whereas large crystals appear much less frequently in the dataset.

6 Pipeline implementation

The complete pipeline has been implemented on Google Colab with a Gradio-based web interface [5] (see Figure 9). This pipeline allows processing of individual images as well as entire image folders.

The complete pipeline executes all the steps described above in a sequential manner. It takes an image as input, detects the crystals and their boundaries, and classifies them into multiple classes. The script outputs a visualization of the crystals on the images, similar to Figures 6 and 7, as well as an Excel file containing the calculated information for each crystal, as illustrated in Table 2. Crystal size measurements are computed according to the conversion table presented in Table 3. This process is summarized in the diagram shown in Figure 8.

Upon execution, the Google Colab script downloads the Detectron2 and GMM models, as well as the `aicrystallmetrics` package that we developed to include utility functions for this project, directly from Google Drive. It sets up the Detectron2 and Gradio environments. The user can then upload either individual images or a zip archive containing the folders crystal images and time images.

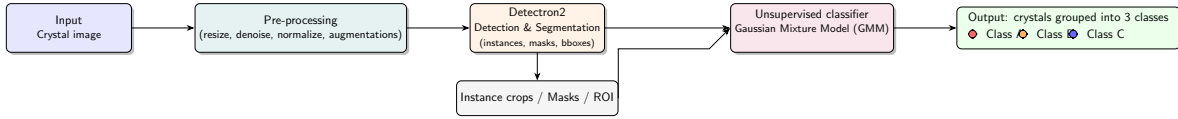


Figure 8: AI pipeline for crystal analysis

crystal_name	size (pixels ²)	size (μm^2)	Class
cristal_001	17421	2073,099	Class C
cristal_002	12486	1485,834	Class C
cristal_003	10653	1267,707	Class C
cristal_004	12525	1490,475	Class C

Table 2: Example of per-image information table

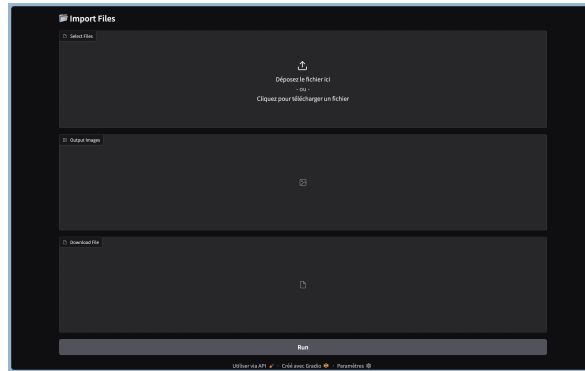


Figure 9: Gradio-Based Web Interface for Crystal Image Detection and Classification

	μm per pixels
Rectangular Images	0.119
Round Images	3.19

Table 3: Pixel-to-Micrometer Conversion Factors for Microscope Images

7 Conclusion

In this project, we have demonstrated a complete implementation of a pipeline for the detection, segmentation, and classification of crystals in microscopic images using Detectron2 and a Gaussian Mixture Model (GMM).

The project deliverables are therefore:

- A dataset annotated with polygons on crystal images.
- Trained models for detection (Detectron2) and classification (GMM).
- A Python package containing all the utility functions needed to manage this project, from training to saving results.
- A web interface implementing the full pipeline to process images from start to finish, outputting crystal sizes and classes in an Excel file.

A potential future objective could be to improve crystal classification by adopting a supervised approach, leveraging metrics such as size or color rather than relying entirely on the model.

Another goal would be to implement the entire pipeline locally, instead of through Google Colab with a web interface as done here, allowing for easier integration with companion microscope software.

References

- [1] K. Wada. Labelme: Image Polygonal Annotation with Python. GitHub repository, 2016. <https://github.com/wkentaro/labelme>
- [2] L. Maier-Hein, A. Reinke, et al. Metrics reloaded: Recommendations for image analysis validation. <https://arxiv.org/abs/2206.01653>
- [3] J. Redmon and A. Farhadi. YOLOv3: An Incremental Improvement. *arXiv preprint arXiv:1804.02767*, 2018. <https://arxiv.org/abs/1804.02767>
- [4] J. Canny. A Computational Approach to Edge Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 8(6):679–698, 1986.
- [5] A. Abid, J. Yu, A. Abdalla, M. Bosma, V. Kuznetsov, and D. Kiela. Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. *arXiv preprint arXiv:1906.02569*, 2019.
- [6] Y. Wu, A. Kirillov, F. Massa, W. Lo, and R. Girshick. Detectron2. <https://github.com/facebookresearch/detectron2>, 2019.
- [7] D. A. Reynolds. Gaussian Mixture Models. https://doi.org/10.1007/978-0-387-73003-5_196, 2009.